Python Programming (BCC-302)

UNIT – 3: Python Complex Data Types

String

Declaration, Operations, Methods

String

- Common datatype which consist of a sequence of letter, character, number, punctuation, and space.
- Immutable and ordered.
- Consider as an array of bytes (each bytes represent a character).
- Python don’t have char datatype.
- Single character is simply a string with one character.

String - Operations

Declaration

- ‘ ‘, “ “, “““ ””” (for multi-line string)
- str() – typecast number or other data types to string.

String Operation

- + (Concatenation
- * (Repetition)
- [:] (Slicing)

String – Operations (cont.)

- **Concatenation**

first = “Sachin"
second = “Tendulkar"
result = first + " " + second
print(result)
O/p: Sachin Tendulkar

- **Repetition**

greet = "Hi! "
print(greet * 3)
O/p: Hi! Hi! Hi!

- **Slicing: [start:stop:step]**

str1="PythonProgramming"
print(str1[0:6])
O/p: Python

print(str1[-6:])
O/p: amming

Practice Questions

1. Write a program to reverse a string.
2. Write a program to find the longest word in a given sentence.
3. Write a program to replace all spaces in a string with hyphens(-).
4. Two strings are anagrams if they contain the same letters. Write a Python program to check.(silent=listen)
5. Write a program to Find and print all words from a sentence that start with a given letter.

List

Defining, Operation, Manipulation

List

- The data type **list** is an ordered sequence that is mutable and made up of one or more elements.
- A list can have elements of different data types, such as integer, float, string, tuple, or even another list.
- A list is very useful to group together elements of mixed data types
- Elements of a list are enclosed in square brackets and are separated by a comma.
- List indices also starts from 0.

List - Operations

Defining

list1 = [20,5.5,'hello']

print(list1)

o/p: [20,5.5,'Hello']

list2=[['Anjali',2000],[''Asha,2001]]

Print(list2)

o/p: [['Anjali',2000],[''Asha,2001]]

Operation

- Accessing list elements
- Concatenation (+)
- Repetition (*)
- Membership (in)
- Slicing [Start, end, index jump]

List – Operations (cont.)

Concatenation

list1=[1,3,5,7,9]

list2=[2,4,6,8,10]

list1+list2

[1,3,5,7,9,2,4,6,8,10]

Repetition

list1=[‘Hello’]

list1*3

[‘Hello’,‘Hello’,‘Hello’]

Membership

list1=[‘Rohan’, ‘Anita’, ‘Manvi’]

‘Manvi’ in list1

True

‘Prashant’ in list1

False

List – Operations (cont.)

- **Slicing**

list1=[‘Riya’, ‘Amit’, ‘Mohan’, ‘Pragya’, ‘Usha’, ’Shreya’]
list1[2:4]

o/p: [‘Mohan’, ‘Pragya’]

List – Operations (cont.)

```
a[start: stop]   # item start through and stops at -1
a[start:]        # item start through the rest of the list
a[: stop]        # item from beginning through stop-1
a[:]             # a copy of whole array
a[-1]            # last item in the array
a[-2:]           # last two item in the array
a[:-2]           # everything except the last two items
a[: :-1]         # all items in the array, reversed
a[1: :-1]        # the first two items , reversed
a[:-3:-1]        # the last two items , reversed
a[-3: :-1]       # everything except the last two items, reversed
```

List - Manipulations

Indexing

```
list1 = [20,5.5,'hello']
print(list1[0])     #20
print(list1[2])     #'hello'

list2=[['Anjali',2000],[''Asha,2001]]
print(list2[0][1])     #2000
print(list2[1][0])     #'Asha'
```

Updating

```
A = [30,60,90,120]
A[3] = 150
print(A)                #[30,60,90150]

A[0] = 80
print(A)                #[80,60,90,150]
```

List - Manipulations

Deleting

```
A = ['Rohit','Rudra','Saloni','Deepak']
del A[3]
print(A)        #['Rohit','Rudra','Saloni']

del A[1]
print(A)        #['Rohit','Saloni']
```

Iterating

```
A = [30,60,90,120]

for i in A:
        print(i)
```

Output:

30
60
90
120

Tuple

Definition, Immutability and use cases

Tuple

- tuple is a sequence of immutable Python objects.
- Tuples are sequences, just like lists.
- The main difference between the tuples and the lists is that the tuples cannot be changed unlike lists. Tuples use parentheses, whereas lists use square brackets .
- Creating a tuple is as simple as putting different comma-separated values. Optionally, you can put these comma-separated values between parentheses also.

Syntax: tuple_name = (item1, item2, item3, ...)

Creating a Tuple

tup1 = ('physics', 'chemistry', 1997, 2000)
o/p: ('physics', 'chemistry', 1997, 2000)

tup2 = (1, 2, 3, 4, 5)
o/p: (1, 2, 3, 4, 5)

tup3 = "a", "b", "c", "d"
o/p: ("a", "b", "c", "d")

tup4 = (5,);
o/p: (5,)

Tuple - Immutability

This means:

- No item assignment
- No item deletion
- No appending or removing elements

Tuple - Operations

- Accessing
- Assignment
- Concatenation
- Repetition
- Membership

Tuple – Operations (cont.)

Accessing

- **Indexing**

tup1 = ('physics', 'chemistry', 2000)

print ("tup1[0]: ", tup1[0])

O/p: tup1[0] : physics

- **Slicing**

tup2 = (1, 2, 3, 4, 5, 6, 7)

print ("tup2[1:5]: ", tup2[1:5])

O/p: tup2[1:5] : [2, 3, 4, 5]

Accessing

- **Basic Unpacking**

tup3 = (200,400,600)

a, b, c=tup3

print(a, b ,c)

O/p: 200 400 600

- **Extended Unpacking**

tup4 = (1, 2, 3, 4, 5)

a, *b, c=tup4

print(a, b ,c)

O/p: 1 [2,3,4] 5

Tuple – Operations (cont.)

Assignment

```
a,b = 3,4
print a  #3
print b  #4

a,b,c = (1,2,3),5,6
print a  #(1, 2, 3)
print b  #5
print c  #6
```

Concatenation

```
a = (20,56,34,68)
b = (10,9,48,36,24)
print( a + b )
```

O/p: (20,56,34,68,10,9,48,36,24)

Tuple – Operations (cont.)

Repetition

('Hi!',) * 4

O/p: Hi!, Hi!, Hi!, Hi!

Length

t1 = (1, 2, 3)

print("Length:", len(t1))

Membership

3 in (1,2,3)

#True

5 in (1,2,3)

#False

Use-Cases

Tuples are used when:

- Data should not change
- Data integrity and safety
- Faster performance than lists
- Representing structured data

Dictionary

Creation, Methods and Manipulation

Dictionary

- Each key is separated from its value by a colon (:), the items are separated by commas, and the whole thing is enclosed in curly braces. An empty dictionary without any items is written with just two curly braces, like this: { }.
- Keys are unique within a dictionary while values may not be. The values of a dictionary can be of any type, but the keys must be of an immutable data type such as strings, numbers, or tuples.

Dictionary

Syntax

```
dictionary_name = {
    key1: value1,     #item1
    key2: value2,    #item2
    ...
}
```

Creation

Using {}

dict() constructor

From list of tuples

Empty dictionary

Dictionary Creation

Using {}

```
student = {
    "name": "Alice",
    "age": 20,
    "course": "B.Tech"
}
print(student)
```

o/p: {'name': 'Alice', 'age': 20, 'course': 'B.Tech'}

Using dict() constructor

```
employee = dict(name="John", id=102, dept="IT")
print(employee)
```

o/p: {'name': 'John', 'id': 102, 'dept': 'IT'}

Dictionary Creation

From list of Tuples

```
data = [('a', 1), ('b', 2), ('c', 3)]
d = dict(data)
print(d)
```

o/p: {'a': 1, 'b': 2, 'c': 3}

Empty Dictionary

```
my_dict = {}
print(my_dict)
```

o/p: ()

Accessing Dictionary Elements

Accessing Items (using keys)

```
dict2 = {'Name': 'Zara', 'Age': 7, 'Class': 'First'}
print ("dict2['Name']: ", dict2['Name'])
print ("dict2['Age']: ", dict2['Age'])
```

o/p

dict2['Name']: Zara

dict2['Age']: 7

Accessing Items (using get())

dict2.get('School' , 'Not Found')

o/p: Not Found

dict2.get(' Class ' , 'Not Found')

o/p: 'First'

Dictionary – Manipulations

Adding Items

```
dict2 = {'Name': 'Zara', 'Age': 7,
'Class': 'First’}

dict2[‘City’]=‘New Delhi’

print ("dict2[‘City']: ", dict2[‘City'])
```

o/p

```
dict2[‘City']: ‘New Delhi’
```

Updating Items

```
dict2 = {'Name': 'Zara', 'Age': 7,
'Class': 'First', 'City’: 'New Delhi’}

dict2[‘Class’]=‘Second’

print ("dict2[‘Class']: ", dict2[‘Class'])
```

o/p

```
dict2[‘Class']: ‘Second’
```

Dictionary – Manipulations

Deleting Items (with del)

```
dict2 = {'Name': 'Zara',
'Age': 7, 'Class': 'First’}

del dict2[‘Class’]
```

o/p

{'Name': 'Zara', 'Age’: 7}

Deleting Items (with pop(key))

```
dict2 = {'Name': 'Zara',
'Age': 7, 'Class': 'First’}

dict2.pop(‘Class’)
```

o/p

‘First'

Deleting Items (with popitem())

```
dict2 = {'Name': 'Zara',
'Age': 7, 'Class': 'First’}

dict2.popitem()
```

o/p

(‘Class’,’First’)

Dictionary Traversal

```
car = {"brand": "Toyota", "model": "Corolla", "year": 2021}

# Keys
for key in car:
    print(key)

# Values
for value in car.values():
    print(value)

# Both key and value
for key, value in car.items():
    print(key, ":", value)
```

o/p:

```
brand
model
year

Toyota
Corolla
2021

brand : Toyota
model : Corolla
year : 2021
```

Built-in Functions

String, List, Dictionary

Built-in Functions: String

Method	**Description**	**Example**
lower()	Converts all characters to lowercase.	s = “Python" print(s.lower()) **o/p:** python
upper()	Converts all characters to uppercase.	s = "python" print(s.upper()) **o/p:** PYTHON
capitalize()	Capitalizes the first letter.	print("java".capitalize()) **o/p:** Java

Built-in Functions: String

Method	**Description**	**Example**
strip()	Removes spaces from both ends.	text = " Hello World " print(text.strip()) **O/p**: Hello World
lstrip()	Removes spaces from the start.	print(text.lstrip()) # **O/p**: Hello World
rstrip()	Removes spaces from the end.	print(text.rstrip()) # **O/p**: Hello World

Built-in Functions: String

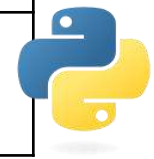
Method	**Description**	**Example**
replace(old, new)	Replaces part of the string.	quote = "Life is short" print(quote.replace("short", "beautiful")) **O/p**: Life is beautiful
split(separator)	Break a string into a list.	data = “apple, banana, cherry" fruits = data.split(",") print(fruits) **O/p**: ['apple', 'banana', 'cherry']
join(list)	Combine a list into a string.	joined = "-".join(fruits) print(joined) **O/p**: apple-banana-cherry

Built-in Functions: String

Method	**Description**	**Example**
find(substring)	Returns index of the first occurrence (or -1 if not found)	msg = "programming" print(msg.find("g")) **O/p**: 3
rfind(substring)	Same as find() but searches from right.	print(msg.rfind("g")) **O/p**: 10
count(substring)	Counts how many times a substring appears.	sentence = "banana banana banana" print(sentence.count("banana")) **O/p**: 3

Built-in Functions: String

Method	Description	Example	Output
len(str)	Find length of string	len("hello")	5
max(str)	Find character with max Unicode value	max("abc")	‘c’
min(str)	Find character with min Unicode value	min("abc")	‘a’
sorted(str)	Returns sorted list of characters based on Unicode/ ASCII value	sorted("cab")	['a', 'b', 'c']
str()	Converts object to string	str(123)	'123'



Built-in Functions: List

Functions	Description	Example
len(list)	Return the length of the list passed as the argument	list1=[10,20,30,40,50] len(list1) **o/p:**5
list()	Creates an empty list if no argument is passed Create a list if sequence is passed as an argument	list1=list() list1 [] str1='aeiou' list1=list(str1) list1 ['a' 'e', 'i', 'o', 'u']

Built-in Functions: List

Functions	Description	Example
append()	Appends a single element passed as an argument at the end of the list. The single element can also be a list.	list1=[10,20,30,40] list1.append(50) list1 [10,20,30,40,50] list1=[10,20,30,40] list1.append([50,60]) list1 [10,20,30,40,[50,60]]

Built-in Functions: List

Functions	Description	Example
extend()	Append each element of the list passed as argument to the end of the given list	list1=[10,20,30] list2=[40,50] list1.extend(list2) list1 [10,20,30,40,50]
insert()	Insert an element at a particular index in the list	list1=[10,20,30,40,50] list1.insert(2,25) list1 [10,20,25,30,40,50] list1.insert(0,5) list1 [5,10,20,25,30,40,50]

Built-in Functions: List

Functions	Description	Example
count()	Return the number of times a given element appears in the list	list1=[10,20,30,10,40,10] list1.count(10) 3
index()	Return index of the first occurrence of the element in the list. If the element is not present, ValueError is generated.	list1=[10,20,30,20,40,10] list1.index(20) 1 list1.index(90) ValueError: 90 is not in list

Built-in Functions: List

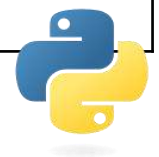
Functions	Description	Example
remove()	Remove the given element from the list. If the element is present multiple times, only the first occurrence is removed. If the element is not present, then Value is generated.	list1=[10,20,30,40,50,10] list1.remove(30) list1=[10,20,40,50,30] list1.remove(90) ValueError: list.remove(x):x not in list
pop()	Removed the element whose index passed as parameter to this function and also remove it from the list. If no parameter is given, then it returns and remove the last element of the list	list1=[10,20,30,40,50,60] list1.pop(3) 40 list1 [10,20,30,50,60]

Built-in Functions: List

Functions	Description	Example
reverse()	Reverse the order of elements in the given list	list1=[34,66,12,89,28,99] list1.reverse() list1 [99,28,89,12,66,34] list1=[‘Tiger’, ‘Lion’, ‘Zebra’, ‘Cat’, ‘Elephant’, ‘Dog’] list1.reverse() list1 [Dog, Elephant, Cat, Zebra, Lion, Tiger]

Built-in Functions: List

Functions	Description	Example
sort()	Sort the element of given list in place	list1=[Tiger, Lion, Zebra, Cat, Elephant, Dog] list1.sort() list1 [Cat, Dog, Elephant, Lion, Tiger, Zebra] list1=[34,66,12,89,28,99] list1.sort(reverse=True) list1 [99,89,66,34,28,12]



Built-in Functions: List

Functions	Description	Example
sorted()	It take a list as parameter and create a new list consisting of the same elements arranged in sorted order	list1=[23,45,11,67,85,56] list2=sorted(list1) list1 [23,45,11,67,85,56] list2 [11,23,45,56,67,85]
min()	Return minimum or smallest element of the list	list1=[34,12,63,39,92,44] min(list1) 12

Built-in Functions: List

Functions	Description	Example
max()	Return maximum or largest element of the list	list1=[34,12,63,39,92,44] min(list1) 92
sum()	Return sum of the elements of the list	list1=[34,12,63,39,92,44] sum(list1) 284

Built-in Functions: Dictionary

Function	Description	Example	Output
len()	Returns no. of elements in a dictionary	d1={1:'A',2:'B',3:'C'} print(len(d1))	3
str()	Returns printable string representation of a dictionary	print(str(d1))	"{1:'A',2:'B',3:'C'}"
clear()	Removes all elements from dictionary	d1.clear	{}
copy()	Returns a shallow copy of dictionary	d2=d1.copy() print(d2)	{1:'A',2:'B',3:'C'}
fromkeys(seq,[,value])	Create a new dictionary with keys from sequence and values set to value • **seq** - This is the list of values which would be used for dictionary keys preparation. • **value** - This is optional, if provided then value would be set to this value	seq = ('name', 'age') d3= d3.fromkeys(seq) print(d3) d3['name']='Ram' print(d3)	{'name': None, 'age': None} {'name': 'Ram', 'age': None}

Built-in Functions: Dictionary

Function	Description	Example	Output
items()	Returns a list of dictionary's (key, value) tuple pairs	d1={'Name':'Nia', 'EmpId':437} d1.items()	dict_items([('Name', 'Nia'), ('EmpId', 437)])
keys()	Returns list of dictionary's keys	d1.keys()	dict_keys(['Name', 'EmpId'])
values()	Returns list of dictionary's values	d1.values()	dict_values(['Nia', 4352])
setdefault(key, default=None)	Returns the key value available in the dictionary and if given key is not available then it will return provided default value. • **key** - This is the key to be searched. • **default** - This is the Value to be returned in case key is not found.	print ("Value : %s" % d1.setdefault('Post', None))	{'Name': 'Nia', 'EmpId': 437, 'Post': None}

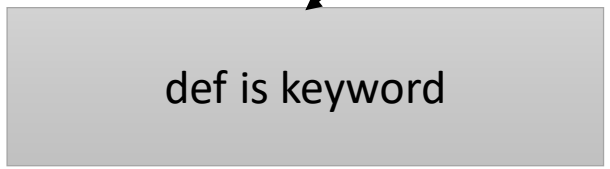
User-defined Functions

User-Defined Functions

Function is a set of statements that performs specific task.

Syntax:

```
def function_name(list of parameters):
    ..............
    ..............
    #function body
    return value     #optional
```



User-defined Functions

1. without parameters
2. with parameters
3. with return value
4. with default Parameters
5. With Arbitrary Arguments (*args & **kwargs)

1. Function without Parameters

```
def greet():
    print("Hello! Welcome to Python functions.")

greet()
```

O/p: Hello! Welcome to Python functions.

2. Function with Parameters

```
def greet_person(name):
    print(f"Hello, {name}! Welcome to Python.")

greet_person("Ansh")
greet_person("Rina")
```

O/p: Hello, Ansh! Welcome to Python.
Hello, Rina! Welcome to Python.

3. Function with return value

```
def add_numbers(a, b):
    return a + b

result = add_numbers(5, 10)
print("Sum:", result)
```

O/p: Sum: 15

4. Function with default parameter

```
def greet_person(name="Guest"):
    print(f"Hello, {name}!")

greet_person()
greet_person(“Jay")
```

O/p: Hello, Guest!
Hello, Jay!

5.1 Function with Arbitrary Arguments (*args)

```
def sum_numbers(*numbers):
    total = 0
    for num in numbers:
        total += num
    return total

print(sum_numbers(1, 2, 3, 4))  # 10
print(sum_numbers(5, 10, 15))   # 30
```

O/p: Hello, Guest!
Hello, Jay!

5.1 Function with Arbitrary Arguments (**kwargs)

```
def student_info(**info):
    for key, value in info.items():
        print(f"{key}: {value}")

student_info(name=“Diya", age=27, course="Python")
```

O/p:
```
name: Diya
age: 27
course: Python
```

Modular Programming

Modular Programming

Module is a container of functions, variables, constants, class in a separate file which can be reused.

Ways to import modules:

i. import statement

ii. from statement

Modular Programming

import statement: used to import entire module.

Syntax: import module name

Example: import math

From statement: imports all functions or selected one.

Syntax: from module name import function name

Example: from random import randint

Modular Programming - Importing in another file

 # mymodule.py

```
def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

 # main.py

```
import mymodule

print(mymodule.add(10, 5))       # 15
print(mymodule.subtract(10, 5))  # 5
```

Modular Programming – Importing Specific function

```
# mymodule.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

```
from mymodule import add

print(add(20, 30))  # 50
```

Modular Programming – Renaming a module

```
# mymodule.py

def add(a, b):
    return a + b

def subtract(a, b):
    return a - b
```

```
import mymodule as mm

print(mm.add(10, 15))  # 25
```

Organizing Python Code using Functions

Basic Function Organization

```
def greet_user(name):
    return f"Hello, {name}!"

def farewell_user(name):
    return f"Goodbye, {name}!"

print(greet_user(“Alice"))
print(farewell_user(“Bob"))
```

O/p: Hello, Alice!
Goodbye, Bob!

Organizing Python code Using Collections – Advanced Containers

Collections – Advanced Containers

1. namedTuple
2. Deque
3. Counter
4. OrderedDict
5. DefaultDict
6. ChainMap

1. Using namedtuple

```
from collections import namedtuple

Student = namedtuple('Student', ['name', 'age', 'course'])

s1 = Student(‘Alice’, 20, 'Python')
s2 = Student(‘Bob', 21, 'Data Science')

print(s1.name)
print(s2.course)
```

Iterating through a namedtuple

```
Student = namedtuple('Student', ['name', 'age', 'course'])
s = Student(‘Pooja', 22, 'Python')

for field in s._fields:
    print(f"{field} = {getattr(s, field)}")
```

Using namedtuple

```
from collections import namedtuple

Student = namedtuple('Student', ['name', 'age', 'course'])
s = Student(‘Alice’, 20, 'Python’)

print(s._asdict())

updated = s._replace(course='Machine Learning')
print(updated)
```

2. Deque (Double-Ended Queue)

```
from collections import deque


dq = deque([1, 2, 3])
dq.append(4)
dq.appendleft(0)
print(dq)         # deque([0, 1, 2, 3, 4])

dq.pop()
dq.popleft()
print(dq)         # deque([1, 2, 3])
```

Deque in Functions

```
from collections import deque

def create_queue(elements):
    return deque(elements)

def enqueue(queue, element):
    queue.append(element)

def dequeue(queue):
   if queue:
      return queue.popleft()
   else:
      return "Queue is empty"
```

```
def display_queue(queue):
   print("Queue:", queue)

q = create_queue([10, 20, 30])
display_queue(q)

enqueue(q, 40)
display_queue(q)

dequeue(q)
display_queue(q)
```

Stack implementation using Deque

```
def create_stack():
    return deque()

def push(stack, item):
    stack.append(item)

def pop_stack(stack):
    if stack:
        return stack.pop()
    else:
        return "Stack is empty"
```

```
def display_stack(stack):
    print("Stack:", stack)

stack = create_stack()
push(stack, "A")
push(stack, "B")
push(stack, "C")
display_stack(stack)

pop_stack(stack)
display_stack(stack)
```

3. Counter

- Counter is a **specialized dictionary** for counting hashable objects.
- It automatically counts elements in iterables

Import it using:

from collections import Counter

Counter - Example

```
from collections import Counter


fruits = ['apple', 'banana', 'apple', 'orange', 'banana', 'apple']
count = Counter(fruits)
print(count)


#Counter({'apple': 3, 'banana': 2, 'orange': 1})
```

Counter Methods

```
from collections import Counter

c = Counter('aabcdeaa')
print(c['a'])                    # Access count of 'a'
c.update('aaabb')                # Add more counts
print(c)
```

O/p:

```
3
Counter({'a': 7, 'b': 3, 'c': 1, 'd': 1, 'e': 1})
```

Counter Methods

```
from collections import Counter

letters = Counter('banana')
print("Most common:", letters.most_common(2))
print("Elements list:", list(letters.elements()))

#Most common: [('a', 3), ('n', 2)]
#Elements list: ['b', 'a', 'a', 'a', 'n', 'n']
```

4. OrderedDict

```
from collections import OrderedDict


od = OrderedDict()
od['a'] = 10
od['b'] = 20
od['c'] = 30


for key, value in od.items():
    print(key, value)
```

O/p:

a 10

b 20

c 30

OrderedDict - Move to End

```
from collections import OrderedDict


od = OrderedDict.fromkeys('abcde')
od.move_to_end('b')
print(od.keys())


od.move_to_end('c', last=False)
print(od.keys())
```

O/p:

```
odict_keys(['a', 'c', 'd', 'e', 'b'])
odict_keys(['c', 'a', 'd', 'e', 'b'])
```

OrderedDict - Comparison with Normal Dict

```
from collections import OrderedDict

od1 = OrderedDict()
od1['x'] = 1
od1['y'] = 2

od2 = OrderedDict()
od2['y'] = 2
od2['x'] = 1

print(od1 == od2)
```

O/p:

False

OrderedDict - Comparison with Normal Dict

Feature	dict	OrderedDict
Preserves insertion order	✓ (since Python 3.7)	✓
Order-based equality check	✗	✓
Extra methods like move_to_end()	✗	✓
Use case	General mapping	Order-sensitive tasks (LRU, logs, etc.)

5. Defaultdict – Eg1 (Avoid KeyError)

```
from collections import defaultdict


dd = defaultdict(int)  # Default value = 0
dd['a'] += 1
dd['b'] += 2
print(dd)


#defaultdict(<class 'int'>, {'a': 1, 'b': 2})
```

Defaultdict – Eg2 (Using list as Default Factory)

```
from collections import defaultdict


dd = defaultdict(list)
dd['fruits'].append('apple')
dd['fruits'].append('banana')
print(dd)


#defaultdict(<class 'list'>, {'fruits': ['apple', 'banana']})
```

Defaultdict – Eg3 (Grouping Words by Length)

```
from collections import defaultdict

words = ['hi', 'hello', 'hey', 'bat', 'ball', 'cat']
group = defaultdict(list)

for word in words:
    group[len(word)].append(word)

print(group)
```

6. ChainMap

```
from collections import ChainMap

defaults = {'theme': 'light', 'language': 'English'}
user_settings = {'theme': 'dark'}

settings = ChainMap(user_settings, defaults)
print(settings['theme'])
print(settings['language'])
```

ChainMap - Eg

```
from collections import ChainMap


a = {'x': 1, 'y': 2}
b = {'y': 3, 'z': 4}
cm = ChainMap(a, b)


print(cm['y'])   # from first map
a['y'] = 99
print(cm['y'])
```

ChainMap - Eg

```
from collections import ChainMap

base = {'a': 1, 'b': 2}
overlay = {'b': 3, 'c': 4}

cm = ChainMap(overlay, base)
print("Original:", cm.maps)

new_cm = cm.new_child({'d': 5})
print("After adding new child:", new_cm.maps)
```

Sets

- Unordered:
- Unique Elements:
- Mutable:
- Immutable Elements

```
 my_set = {1, 2, 3, 4, 2}
print(my_set)
# Output: {1, 2, 3, 4} (duplicates are removed)
```

Frozenset

- Immutable
- Elements cannot be changed
- hashable

Syntax:

frozenset([iterable])

Frozenset - Eg

```
A = frozenset([1, 2, 3, 4])
B = frozenset({3, 4, 5, 6})
print("A =", A)
print("B =", B)
```

O/p:

```
A = frozenset({1, 2, 3, 4})
B = frozenset({3, 4, 5, 6})
```

Frozenset - Operations

```
A = frozenset([1, 2, 3, 4])
B = frozenset([3, 4, 5, 6])

print("Union:", A | B)
print("Intersection:", A & B)
print("Difference:", A - B)
print("Symmetric Difference:", A ^ B)
```

O/p:

```
Union: frozenset({1, 2, 3, 4, 5, 6})
Intersection: frozenset({3, 4})
Difference: frozenset({1, 2})
Symmetric Difference: frozenset({1, 2, 5, 6})
```

Frozenset - as a dictionary key

```
d = {}
key1 = frozenset([1, 2, 3])
key2 = frozenset([4, 5])

d[key1] = "Group 1"
d[key2] = "Group 2"

print(d)
```

O/p:

```
{frozenset({1, 2, 3}): 'Group 1', frozenset({4, 5}): 'Group 2'}
```

Frozenset - Invalid operations

```
f = frozenset([1, 2, 3])
f.add(4)      # Error
f.remove(2)   # Error


print(d)
```

O/p:

```
AttributeError: 'frozenset' object has no attribute 'add'
```

Array Module

- The **array module** provides a **space-efficient way to store large sequences of numeric values**.
- Unlike lists, arrays can **store only one data type** (e.g., all integers or all floats).

import array

array.array(typecode, [initializer])

Array Module – Common type codes

Type Code	C Type	Python Type	Bytes
`'b'`	signed char	int	1
`'B'`	unsigned char	int	1
`'i'`	signed int	int	2/4
`'I'`	unsigned int	int	2/4
`'f'`	float	float	4
`'d'`	double	float	8

Array Module

```
import array

arr = array.array('i', [10, 20, 30, 40])
print("Array elements:", arr)

print("First element:", arr[0])
print("Last element:", arr[-1])
```

O/p:

```
Array elements: array('i', [10, 20, 30, 40])
First element: 10
Last element:40
```

Array Module

```
import array

arr = array.array('i', [10, 20, 30, 40])

arr.append(50)

arr.extend([60, 70])

arr.insert(2, 25)   # insert at index 2
print(arr) #array('i', [10, 20, 25, 30, 40, 50, 60, 70])
```

Array Module

```
import array


arr = array.array('i', [10, 20, 25, 30, 40])
arr.remove(25)     # Removes first occurrence of 25
print("After remove:", arr)

popped = arr.pop()  # Removes last element
print("Popped element:", popped)
print("After pop:", arr)
```

Array Module

```
import array


arr = array.array('i', [10, 20, 30, 40])
for x in arr:
    print(x, end=" ")



#10 20 30 40 50 60
```

Array Module

```
import array


arr = array.array('i', [10, 20, 30, 40])
print(arr[1:4])



#array('i', [20, 30, 40])
```

Array Module

```
import array

arr = array.array('i', [10, 20, 30, 40])
list_data = arr.tolist()       # Array → List
print("List:", list_data)

new_arr = array.array('i', list_data)  # List → Array
print("Array:", new_arr)
```

O/p:

```
List: [10, 20, 30, 40, 50, 60]
Array: array('i', [10, 20, 30, 40, 50, 60])
```

Queue Module

- provides a **thread-safe** way to handle queues
- mainly used in **multithreaded programming**

Importing Module:

import queue

Types of Queues

Type	Class	Description
FIFO Queue	queue.Queue	First In, First Out order
LIFO Queue	queue.LifoQueue	Last In, First Out (like a stack)
Priority Queue	queue.PriorityQueue	Elements are retrieved based on priority

Common Methods

Method	Description
q.put(item)	Adds (enqueues) an item to the queue
q.get()	Removes (dequeues) an item from the queue
q.qsize()	Returns the number of items in the queue
q.empty()	Returns True if queue is empty
q.full()	Returns True if queue is full
q.put_nowait(item)	Adds item without blocking
q.get_nowait()	Gets item without blocking

FIFO Queue

```
import queue

# Create a FIFO queue
q = queue.Queue()

# Add elements
q.put(10)
q.put(20)
q.put(30)

print("Queue size:", q.qsize())
```

```
# Remove elements
print("Removed item:",
q.get())
print("Removed item:",
q.get())

# Remaining items
print("Queue empty?",
q.empty())
```

LIFO Queue

```
import queue

# Create a LIFO queue
stack = queue.LifoQueue()

# Add elements
stack.put('A')
stack.put('B')
stack.put('C')

print("Stack size:", stack.qsize())

# Remove elements
print("Removed item:", stack.get())
print("Removed item:", stack.get())
print("Removed item:", stack.get())

# Check empty
print("Stack empty?", stack.empty())
```

Priority Queue

```
import queue


# Create a Priority Queue
pq = queue.PriorityQueue()


# Add elements (priority,
value)
pq.put((3, 'Low priority'))
pq.put((1, 'High priority'))
pq.put((2, 'Medium priority'))
```

```
# Retrieve elements
print(pq.get())
print(pq.get())
print(pq.get())
```

Itertools Module

To import:

import itertools

Function:

- Product()
- Permutations()
- Combinations()
- Groupby()

1. itertools.product()

Returns the **Cartesian product** of input iterables.

Syntax:

itertools.product(iter1, iter2, ..., repeat=1)

Examples of itertools.product()

```
import itertools
a = [1, 2]
b = ['x', 'y']
result = itertools.product(a, b)
print(list(result))

#[(1, 'x'), (1, 'y'), (2, 'x'), (2, 'y')]
```

```
import itertools
result = itertools.product([0, 1], repeat=3)
print(list(result))

#[(0, 0, 0), (0, 0, 1), (0, 1, 0), (0, 1, 1),
 (1, 0, 0), (1, 0, 1), (1, 1, 0), (1, 1, 1)]
```

2. itertools.permutations()

- Generates **all possible orderings** of the given elements.

Syntax:

itertools.permutations(iterable, r=None)

Examples of itertools.permutations()

```
import itertools

data = [1, 2, 3]
result = itertools.permutations(data)
print(list(result))
```

O/p:

```
[(1, 2, 3), (1, 3, 2),
 (2, 1, 3), (2, 3, 1),
 (3, 1, 2), (3, 2, 1)]
```

```
import itertools

result = itertools.permutations('ABC', 2)
print(list(result))
```

O/p:

```
[('A', 'B'), ('A', 'C'),
 ('B', 'A'), ('B', 'C'),
 ('C', 'A'), ('C', 'B')]
```

3. itertools.combinations()

- Generates all possible **combinations** of a given length.

Syntax:

itertools.combinations(iterable, r)

Examples of itertools.combinations()

```
import itertools

data = [1, 2, 3]
result = itertools.combinations(data, 2)
print(list(result))
```

O/p:

[(1, 2), (1, 3), (2, 3)]

```
import itertools

result = itertools.combinations('ABC', 2)
print(list(result))
```

O/p:

[('A', 'B'), ('A', 'C'), ('B', 'C')]

4. itertools.groupby()

- Groups elements from an iterable **based on a key function**.

Syntax:

itertools.groupby(iterable, key=None)

Examples of itertools.groupby()

```
import itertools

data = [("fruit", "apple"),
        ("fruit", "banana"),
        ("vegetable", "carrot"),
        ("vegetable", "onion")]

for key, group in itertools.groupby(data, lambda x: x[0]):
    print(key, list(group))
```

O/p:

```
fruit [('fruit', 'apple'), ('fruit', 'banana')]
vegetable [('vegetable', 'carrot'),
('vegetable', 'onion')]
```

Examples of itertools.groupby()

```
import itertools

nums = [1, 1, 2, 2, 3, 3, 4]
for key, group in itertools.groupby(nums, key=lambda x: x % 2 == 0):
    print(key, list(group))
```

O/p:

False [1, 1]

True [2, 2]

False [3, 3]

True [4]

Exceptions

Exception	Description
ZeroDivisionError	Raised when dividing by zero
TypeError	Invalid operation between different data types
ValueError	Invalid value (e.g., converting "abc" to int)
IndexError	Index out of range in a list or string
KeyError	Accessing a dictionary with a non-existing key
NameError	Using a variable that’s not defined
FileNotFoundError	When a file operation fails due to missing file
AttributeError	Accessing an invalid attribute of an object

Exceptions -Eg

```
try:
    a = int(input("Enter number: "))
    b = int(input("Enter another number: "))
    print("Result:", a / b)
except ZeroDivisionError:
    print("You cannot divide by zero!")
except ValueError:
    print("Please enter valid numbers!")
finally:
    print("Execution completed.")
```

O/p:

Enter number: 5
Enter another number: 0
You cannot divide by zero!
Execution completed.

Exceptions -Eg

```
try:
    num = int(input("Enter a positive number: "))
    if num < 0:
        raise ValueError("Negative number not allowed")
except ValueError as e:
    print("Error:", e)
else:
    print("Square:", num ** 2)
finally:
    print("Done!")
```

O/p:

Enter a positive number: -3

Error: Negative number not allowed

Done!

Exceptions -Eg

```
class UnderAgeError(Exception):
    pass

try:
    age = int(input("Enter your age: "))
    if age < 18:
        raise UnderAgeError("You must be 18 or older.")
    print("Access granted.")
except UnderAgeError as e:
    print("Error:", e)
```

O/p:

Enter your age: 3

Error: You must be 18 or older

Enter your age: 20

Access granted